

Chapter 5

程式撰寫的演進

Evolution of Programming

Horazon / Antigravity

本章大綱 (Agenda)

本章將帶領你了解寫程式的方法是如何演進的：

1. **早期階段**：機器碼與組合語言
2. **高階語言**：純手工撰寫程式的時代
3. **Tab Coding**：AI 輔助的自動補全
4. **Vibe Coding**：透過對話與同步驗證開發
5. **Agent Coding**：代理程式同步協作
6. **自動化 Agent**：自主規劃與執行的未來

1. 早期階段：與硬體直接溝通

流行時間：1940s ~ 1950s

代表工具：打孔卡片 (Punched Cards)、早期大型主機

機器碼 (Machine Code)

- 直接輸入 `0` 和 `1` 讓電腦執行，需要極高的硬體知識。

組合語言 (Assembly Language)

- 將二進位指令轉換為簡單指令 (如 `ADD`)，稍微提升了閱讀性。

2. 高階語言：純手工撰寫

流行時間：1960s ~ 2020s

代表工具：C, Java, Python / Notepad++, Visual Studio

讓程式更像人類語言

- 工程師發明了高階語言，語法更加接近英文邏輯。

逐字敲擊程式碼的時代

- 程式設計師必須從頭到尾**純手工撰寫**每一行程式碼。
- 尋找錯誤 (Bug) 與查閱文件 (如 StackOverflow) 需要耗費大量時間。

3. Tab Coding：自動補全程式碼

流行時間：2021 ~ 今日

代表工具：GitHub Copilot, Tabnine, AWS CodeWhisperer

智慧提示 (Intelligent Code Completion)

- 編輯器 (如 VS Code) 提供建議，打幾個字就能列出可能的選項。

AI 輔助的崛起

- AI 能猜測你接下來要寫的邏輯。
- 工程師只需按下 **Tab** 鍵，就能**自動補全一整段程式碼**，大幅提升速度。

4. Vibe Coding：同步驗證

流行時間：2023 ~ 今日

代表工具：ChatGPT, Cursor, Claude

對話式開發 (Conversational Coding)

- 將問題丟給 AI，由 AI 負責提供完整的程式碼解決方案。

邊做邊驗證

- 工程師把重心放在**「概念溝通」與「檢查結果」**。
- 將 AI 產出貼上後立刻執行與驗證，如有錯誤再將資訊丟回給 AI 修正。

5. Agent Coding：同步協作

流行時間：2024 ~

代表工具：Cursor Composer, GitHub Copilot Workspace, Windsurf

專屬的 AI 助理

- AI 不只會回答問題，還能像助手一樣在編輯器旁隨時待命。

同步處理複雜任務

- 提供工具權限，讓 AI Agent 可以**同時讀取多個檔案、分析架構**。
- 工程師負責審核與裁決，讓 Agent 直接幫忙寫入與修改專案檔案。

6. 自動化 Agent：完全自主

流行時間：2025 ~ 未來

代表工具：Google Antigravity, Claude

從輔助到自主執行 (Autonomous AI)

- 進階的 Agent 能自動理解大型需求，不再依賴人類頻繁下指令。

端到端的工作流程

- 告訴它目標後，Agent 會：
 - i. 自主拆解步驟與規劃任務。
 - ii. 自動修改檔案、安裝套件並處理終端機。
 - iii. 自動偵測修復錯誤與回報進度。
- 真正意義上的「自動化開發」。

Agent 工具分類 (2025年最新)

目前市面上的 Agent 根據「操作介面」主要分為兩大陣營：

1. 終端機模式 (CLI Agents)

- Claude Code (Anthropic)
- Gemini CLI (Google)
- Aider / ForgeCode (Open Source)

2. 視覺開發環境模式 (Agentic IDEs / UI)

- Antigravity (Google)
- Cursor / Windsurf (Anysphere / Codeium)
- GitHub Copilot "Agent Mode" (Microsoft)
- JetBrains Junie (JetBrains)

Horazon / Antigravity Lesson 05

Thank You

Any Questions?